

깃허브 액션으로 만드는 24시간 클라우드 자동화

Claude Cowork*Skill

책의 6장에서는 코워크의 작업 예약 기능을 사용해 반복 작업을 자동화해 봤습니다. 간단한 자동화라면 코워크 기능만으로도 충분하지만, 더 다양한 서비스와 연동하거나 컴퓨터를 꺼도 장기간 안정적으로 운영하려면 깃허브 액션 [GitHub Actions](#)이 적합합니다. 이번에는 data-collector 스킬에 포함된 기능을 활용해 깃허브 액션으로 자동화하는 방법을 알아보겠습니다.

1 자동화 전 준비

코워크 스케줄만 사용할 계획이라면 별도의 준비는 필요하지 않습니다. 하지만 깃허브 액션과 슬랙 [Slack](#) 알림까지 함께 사용하려면 다음 두 가지를 미리 준비해 두면 됩니다. 지금 당장 준비하지 않아도 괜찮습니다. 실습을 먼저 읽은 뒤 실제 자동화를 시작할 때 다시 돌아와 따라 해도 됩니다.

1.1 깃허브 계정

깃허브 [GitHub](#)는 코드를 저장하고 관리하는 서비스입니다. 이번 실습에서는 코드를 보관하는 것보다 깃허브 액션이라는 자동 실행 기능을 활용하는 것이 목적입니다. 마치 구글 드라이브에 파일을 올려두면 공유가 되는 것처럼, 깃허브에 코드를 올려두면 자동 실행이 되는 것입니다.

무료 계정만으로도 충분하며, 깃허브 액션도 월 2,000분까지 무료로 사용할 수 있습니다. 하루 한 번 정도 보고서를 생성하는 수준이라면 무료 범위 안에서 넉넉하게 사용할 수 있습니다. 가입은 [github.com](#)에서 이메일과 사용자 이름, 비밀번호만 입력하면 됩니다.

1.2 슬랙 웹훅 URL

이번 실습에서는 보고서가 완성되면 슬랙으로 자동 알림을 보내는 기능도 함께 구현합니다. 따라서 슬랙 웹훅 [Slack Webhook](#) URL도 함께 준비합니다. 먼저 이 책 본문에서와 같이 슬랙 워크스페이스와 채널을 만들어 둡니다. 이 책에서는 #trend-report라는 채널을 만들었습니다.

Sign up for GitHub

Continue with Google

Continue with Apple

or

Email*

✓

Password*

✓

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

✓

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region*

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

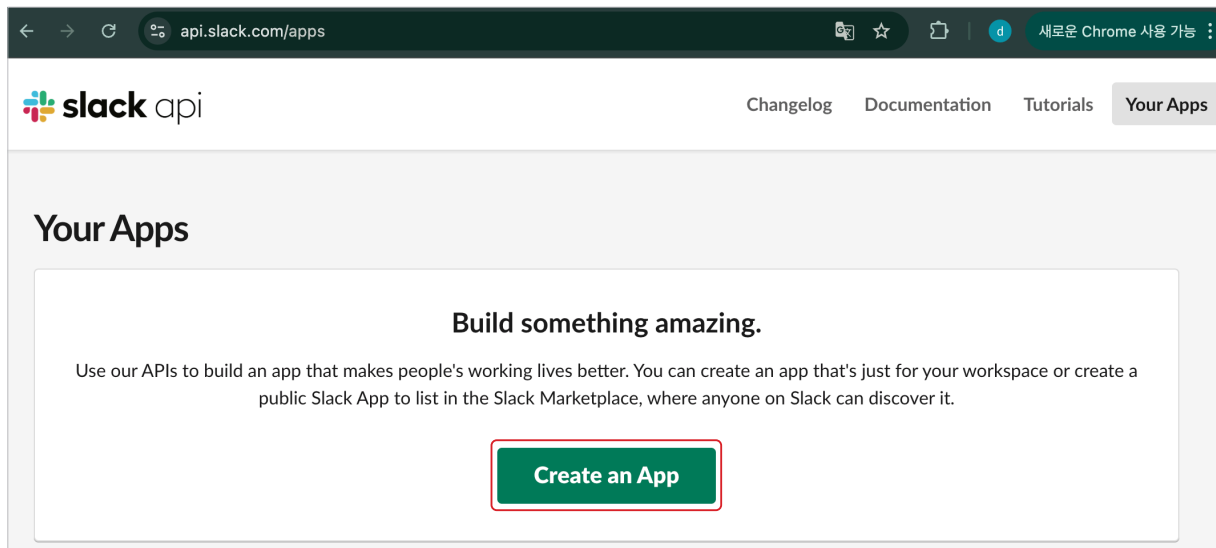
Create account >

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

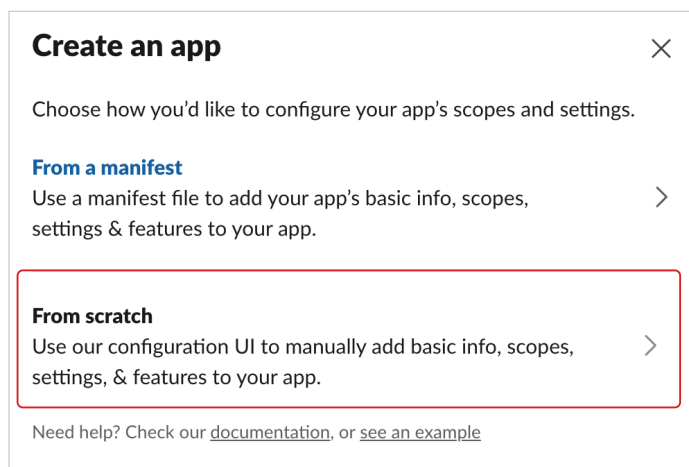
github.com 가입 화면(이메일, 사용자 이름, 비밀번호 입력)

이제 슬랙이 외부에서 메시지를 받을 수 있도록 웹훅(Webhook)을 생성합니다. 슬랙 웹훅은 외부 프로그램이 슬랙으로 메시지를 보내기 위한 전용 주소입니다. 우체통에 편지를 넣으면 집으로 전달되는 것처럼 이 주소로 데이터를 보내면 지정한 채널에 자동으로 메시지가 나타납니다. 설정은 다음 순서대로 진행합니다.

Step 1 브라우저에서 api.slack.com/apps에 접속한 뒤 [Create an App]을 클릭합니다.



Step 2 이어서 열리는 창에서 [From scratch]를 선택합니다.



슬랙에서는 Manifest 방식과 From scratch 방식을 제공합니다. Manifest는 미리 작성된 설정 파일을 이용해 복잡한 앱을 만드는 방식이고, From scratch는 필요한 기능만 직접 설정하는 방식입니다.

이번 실습에서는 웹훅만 사용하는 간단한 앱이므로 [From scratch]를 선택하면 됩니다.

Step 3 간단한 앱 이름을 지정하고 앞서 만든 워크스페이스를 선택한 뒤 [Create App]을 클릭합니다.

Name app & choose workspace

App Name

23

Don't worry - you'll be able to change this later. Note that app names do not allow certain special characters; only a limited number are permitted. For example, a dash or parentheses.

Pick a workspace to develop your app in:

Team Project

Keep in mind that you can't change this app's workspace later. If you leave the workspace, you won't be able to manage any apps you've built for it. The workspace will control the app even if you leave the workspace.

[Sign into a different workspace](#)

By creating a Web API Application, you agree to the [Slack API Terms of Service](#).

Cancel

Create App

Step 4 앱이 만들어지면 나오는 다음과 같은 화면에서 왼쪽 메뉴의 [Incoming Webhooks]를 클릭합니다.

slack api

Changelog
Documentation

Demo App

Basic Information

App Credentials

These credentials allow your app to access the Slack API. They are secret. Please don't share your app credentials with anyone, include them in public code repositories, or store them in insecure ways.

App ID

Date of App Creation

Client ID

Client Secret

Signing Secret

Verification Token

Settings

Basic Information

Collaborators

Socket Mode

Install App

Manage Distribution

Features

App Home

Agents & AI Apps

Work Object Previews...

Workflow Steps

Org Level Apps

Incoming Webhooks

Interactivity & Shortc...

Slash Commands

Steps from Apps

OAuth & Permissions

Event Subscriptions

User ID Translation

App Manifest

Beta Features

Submit to Slack Marketplace

Review & Submit

Step 5 그림처럼 토글을 활성화하고 [Add New Webhook]을 클릭합니다.

Activate Incoming Webhooks

Incoming webhooks are a simple way to post messages from external sources into Slack. They make use of normal HTTP requests with a JSON payload, which includes the message and a few other optional details. You can include [message attachments](#) to display richly-formatted messages.

Adding incoming webhooks requires a bot user. If your app doesn't have a [bot user](#), we'll add one for you.

Each time your app is installed, a new Webhook URL will be generated.

If you deactivate incoming webhooks, new Webhook URLs will not be generated when your app is installed to your team. If you'd like to remove access to existing Webhook URLs, you will need to [Revoke All OAuth Tokens](#).

☒ On

Webhook URLs for Your Workspace

To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an `application/json` POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

```
curl -X POST -H 'Content-type: application/json' --data '{"text":"Hello, World!"}' YOUR_WEBHOOK_URL_HERE
```

Webhook URL	Channel	Added By
No webhooks have been added yet.		

Add New Webhook

Step 6 마지막으로, 메시지를 받을 채널을 선택하는 화면이 나타납니다. 앞서 만들어둔 trend-report 채널을 선택하고 [허용]을 누릅니다.

"Demo App" 앱의 Slack 액세스 허용

이 앱은 고객님의 워크스페이스인 dololo의 멤버가 만들었습니다.

Workspace

웹훅크를 위한 채널

채널 검색...

채널

새-워크스페이스-전체

새-채널

소셜

trend-report

Review app permissions

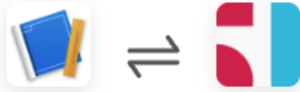
Slack은 귀하에게 부여된 권한을 Demo App 앱과 공유하게 됩니다. Slack의 [개인정보 처리방침](#)을 확인하세요. Slack이 귀하에게 부여된 권한을 공유하도록 허용하는 데에 동의하는 경우, Demo App의 개인 정보 보호 계약 및 서비스 약관에도 함께 동의하게 됩니다. 설치하기 전에 Demo App의 약관을 검토하세요.

취소

허용



만약 채널 선택 화면 대신 “앱에 설치할 봇 사용자가 없습니다”라는 메시지가 표시된다면 추가 설정이 필요합니다.



데이터 수집 알림 앱에 설치할 봇 사용자가 없습니다.

이 앱은 워크스페이스에 봇을 설치할 권한을 요청하지만 현재는 봇으로 구성되어 있지 않습니다. 이 문제를 해결하려면 [개발자에게 문의](#)해주세요.

[데이터 수집 알림 앱으로 돌아가기](#)

1. 앞서 Step 4의 왼쪽 메뉴에서 [OAuth & Permissions]를 클릭합니다. 페이지를 아래로 스크롤하면 Bot Token Scopes 섹션이 보입니다. 여기에 incoming-webhook이 이미 추가되어 있는지 확인합니다. 없다면 [Add an OAuth Scope] 버튼을 클릭한 뒤, incoming-webhook을 검색해서 선택합니다.
2. 왼쪽 메뉴에서 [App Home]을 클릭합니다. Your App's Presence in Slack 영역 안에 App Display Name 섹션이 있습니다. [Edit] 버튼을 클릭하고, [Display Name]에 ‘트렌드 수집봇’처럼 알아보기 쉬운 이름을 입력합니다. [Default Username]도 ‘trend-bot’ 같은 영문으로 입력한 뒤 [Save]를 누릅니다. 슬랙은 봇 이름이 등록되어야 앱을 봇으로 인식하므로 이 단계가 핵심입니다.
3. 왼쪽 메뉴에서 [Incoming Webhooks]로 돌아가 다시 [Add New Webhook]을 클릭합니다. 그러면 정상적으로 채널 선택 화면이 나타납니다.

이제 앞에서 정한 채널을 선택하고 [허용]을 누르면 됩니다.

Step 7 이렇게 채널을 선택하고 나면 Webhook URL이 생성됩니다. [Copy] 버튼을 눌러 복사해 두세요. 나중에 클로드에게 알려 주거나 깃허브에 설정값으로 추가할 때 사용합니다.

Webhook URLs for Your Workspace

To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an [application/json](#) POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

```
curl -X POST -H 'Content-type: application/json' --data '{"text":"Hello, World!"}' https://hooks.slack.com/services/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
```

[Copy](#)

Webhook URL	Channel	Added By
https://hooks.slack.com/services/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 Copy	#trend-report	Live Jun 17, 2026

[Add New Webhook](#)

여기서 생성한 웹훅 URL은 잘 보관해 두고 비밀번호처럼 취급해야 합니다. 이 값을 아는 사람은 누구든 해당 채널에 메시지를 보낼 수 있기 때문입니다.

이제 준비가 끝났으니 본격적으로 자동화를 요청해 보겠습니다.

2 자동화 요청하기

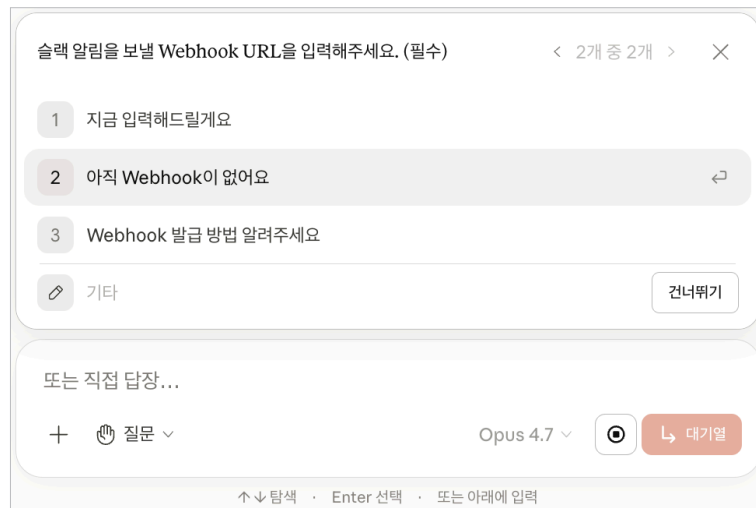
이 책 본문의 두 번째 실습으로 만든 보고서가 마음에 들었다면, 이어서 /data-collector 스킬을 명시적으로 호출하면서 다음과 같이 말해 보세요.

/data-collector 이 분석을 매일 자동으로 돌리고 싶어. 매일 아침 9시에 실행되게 해줘. 결과가 나오면 슬랙으로 알림도 보내줘.

앞선 대화가 이어지고 있으므로 분석 주제를 다시 설명할 필요는 없습니다. 클로드는 방금 수행한 반도체 시장 분석을 그대로 이어서 자동화 작업을 준비합니다.

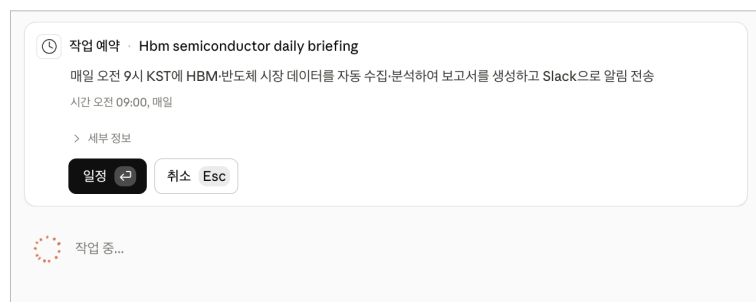
자동화 환경 선택을 묻는 질문

자동화를 시작하면 클로드가 어떤 방식으로 구성할지 먼저 묻습니다. 코워크 스케줄과 깃허브 액션을 모두 선택하면 두 가지 자동화 환경을 한 번에 만들어 줍니다.



슬랙 웹훅 URL 관련 질문

이어지는 화면에서 슬랙 웹훅 URL을 요청할 수 있습니다. 우리는 나중에 따로 GitHub Secrets로 입력할 예정이므로 여기서 입력하지는 않고 건너뛴다.



코워크 스케줄과 자동화 코드 생성 중 화면

잠시 후 작업이 완료되면 깃허브 액션 등록에 필요한 ZIP 파일도 하나 생성됩니다. 압축을 풀어 보면 여러 개의 파일이 들어 있는데, 각각 다음과 같은 역할을 담당합니다.

- **run_pipeline.py**: 전체 자동화 파이프라인을 실행하는 메인 파일입니다. 데이터 수집 → 분석 → 보고서 생성 → 슬랙 알림 순서로 모든 작업을 실행합니다. 여러분이 이 파일 하나만 실행하면 나머지는 알아서 돌아갑니다.
- **collector.py**: RSS 피드와 뉴스 API에서 데이터를 수집합니다. 이 코드는 클로드 없이도 혼자서 실행될 수 있습니다.
- **analyzer.py**: 수집한 데이터를 분석하고 핵심 키워드와 트렌드를 추출합니다.
- **report_generator.py**: 분석 결과를 사람이 읽기 좋은 보고서(예를 들어 마크다운)로 변환합니다.
- **slack_notifier.py**: 보고서가 완성되면 슬랙 채널에 알림을 보냅니다. 앞에서 만든 웹훅 URL을 통해 메시지가 전송됩니다. 이 코드는 깃허브 액션의 환경 변수(Secrets, 코드 바깥에서 안전하게 주입되는 비밀 설정값)에서 웹훅 URL을 먼저 읽고, 정보가 없으면 config.yaml에서 읽도록 설계되어 있습니다. 따라서 config.yaml에 URL을 직접 넣지 않아도 깃허브에서 정상으로 동작합니다.
- **config.yaml**: 전체 파이프라인의 설정을 담고 있습니다. 분석 키워드와 RSS 주소, 보고서 저장 위치 등이 적혀 있습니다. 나중에 분석 키워드를 바꾸고 싶으면 이 파일만 수정하면 됩니다. 슬랙 웹훅 URL 칸은 비어 있는 것이 정상입니다. 여기에 직접 URL

을 넣으면 깃허브에 올렸을 때 URL이 외부로 노출될 수 있으니, 반드시 비워 두고 GitHub Secrets에 따로 등록합니다.

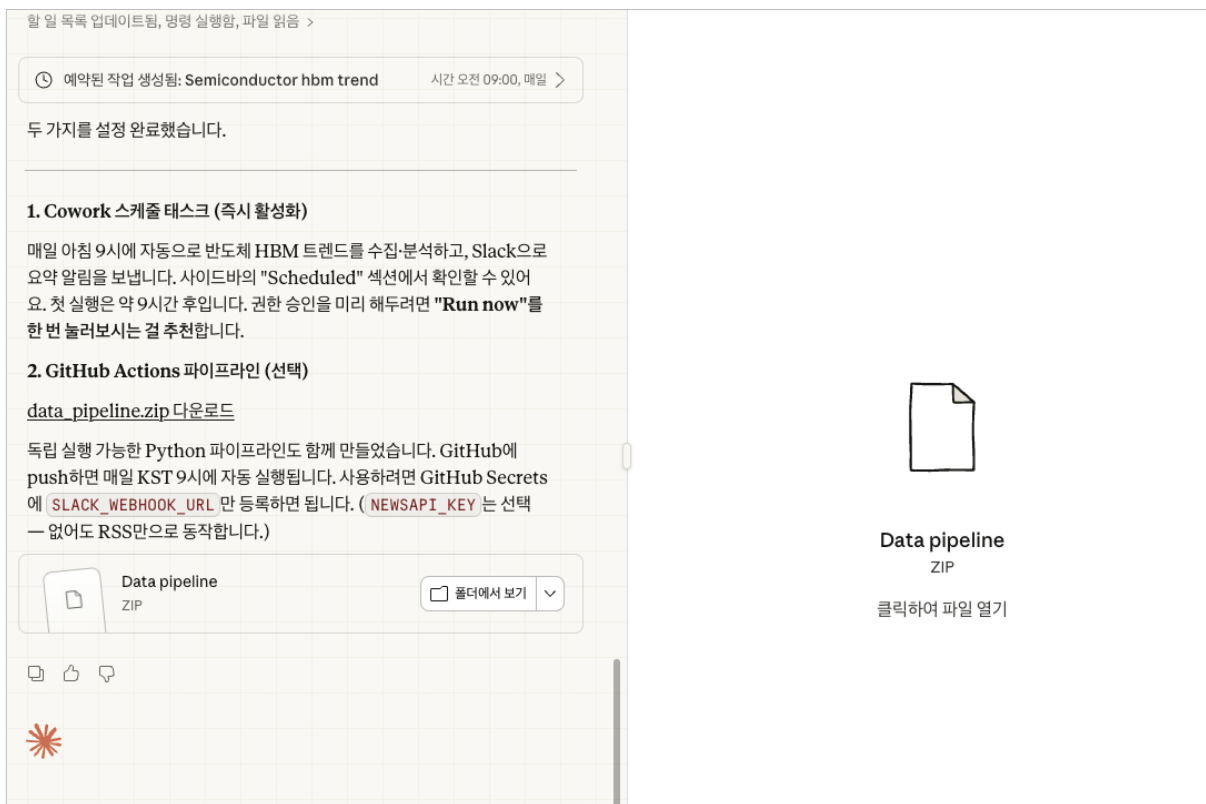
- **requirements.txt**: 코드 실행에 필요한 라이브러리 목록입니다.
- **README.md**: 설치와 실행 방법을 설명합니다.
- **.github/workflows/daily_collect.yml**: 이 파일이 깃허브 액션에게 "매일 아침 9시에 run_pipeline.py를 실행해 줘"라고 알려주는 지시서입니다. 마치 알람 시계를 설정해 두는 것과 비슷합니다. 이 파일이 없으면 깃허브에 업로드해도 자동 실행이 되지 않으니, ZIP 파일을 풀었을 때 .github 폴더가 있는지 반드시 확인하세요.

Tip

ZIP 파일을 풀었을 때 .github 폴더가 없다면, 클로드에게 다음과 같이 요청하면 됩니다.

GitHub Actions 워크플로(workflow, 정해진 순서대로 작업을 자동 실행하는 절차서) 파일도 만들어줘. 매일 KST 오전 9시에 run_pipeline.py를 실행하는 yml 파일이야.

그러면 클로드가 daily_collect.yml 파일을 추가로 만들어 줍니다. 쉽게 말하면, 코워크에서 여러분이 직접 시키던 일을 깃허브가 매일 대신 수행하는 구조입니다. run_pipeline.py가 컨베이어 벨트, 나머지 파일이 각 공정의 기계라고 생각하면 됩니다.



완성된 ZIP 파일이 코워크 오른쪽에 표시된 화면

자동화 구성이 완료되면 두 가지 결과가 동시에 준비됩니다.

첫 번째는 코워크 스케줄입니다. 코워크 안에서 매일 오전 9시에 자동으로 실행되며, 사이드바의 [예정됨] 메뉴에서 실행 상태를 확인할 수 있습니다.

두 번째는 깃허브 액션 파이프라인입니다. 생성된 ZIP 파일을 깃허브 저장소에 업로드하면 클라우드에서 같은 작업을 자동으로 실행할 수 있습니다. 슬랙 웹훅 URL은 GitHub Secrets에 등록하면 되고, NewsAPI 키가 없어도 RSS 기반으로 기본 동작은 가능합니다.

즉, 같은 자동화를 코워크 안에서 실행할 수도 있고, 깃허브를 이용해 독립적으로 운영할 수도 있습니다. 상황에 맞는 방식을 선택하면 됩니다.

2.1 자동화를 실행하는 두 가지 방법

앞에서 클로드가 자동화 파일(ZIP)과 함께 코워크 스케줄도 등록해 준다고 설명했습니다. 사실 “매일 자동으로 실행해 줘”라고 요청하면 클로드는 두 가지 실행 방법을 함께 준비합니다.

구분	코워크 스케줄	깃허브 액션
설정	클로드가 자동으로 등록	ZIP을 GitHub에 업로드
추가 가입	없음	깃허브 계정 필요
PC를 꺼도 실행	불가능	가능(클라우드 실행)
비용	무료	무료(월 2,000분)
적합한 상황	개인 업무, PC를 항상 켜두는 경우	24시간 안정 실행, 팀 공유

둘 중 하나만 사용해도 되고, 둘을 함께 사용해도 됩니다. 예를 들어 평소에는 코워크 스케줄을 이용하다가 출장이나 휴가처럼 컴퓨터를 오래 끄는 기간에는 깃허브 액션이 대신 실행하도록 구성할 수도 있습니다.

이제 깃허브 액션을 설정하는 방법을 차례대로 살펴보겠습니다.

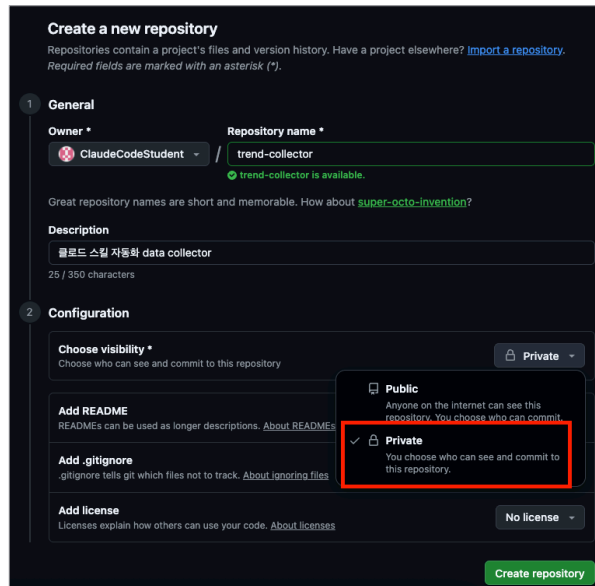
2.2 깃허브에 올리기

앞에서 안내하는 대로 깃허브 계정을 만들어 두었다면 이제 클로드가 만든 파일을 업로드하면 됩니다.

Step 1 새 저장소 만들기

깃허브에 로그인한 뒤 오른쪽 위 [+] → [New repository]를 클릭합니다. 저장소 이름은 자유롭게 정하면 되지만, 나중에 알아보기 쉽도록 ‘trend-collector’나 ‘data-pipeline’처럼 프로젝트를 설명하는 이름을 추천합니다.

공개 여부는 [Private]을 선택하세요. 분석 결과나 설정 파일을 공개할 이유가 없다면 Private이 안전합니다. 나머지 옵션은 그대로 두고 [Create repository]를 클릭합니다.

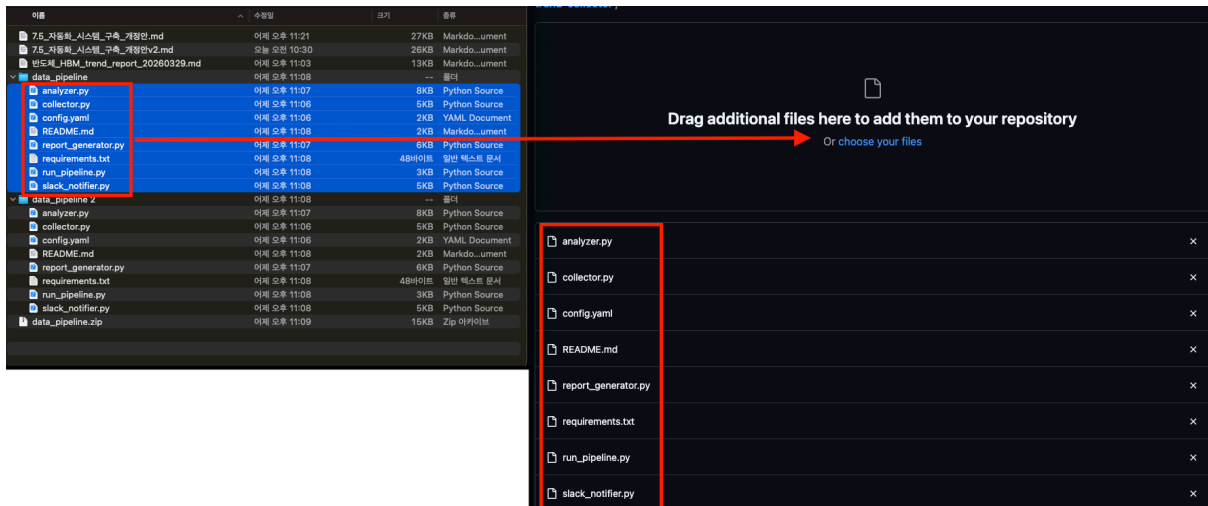


깃허브 저장소 생성

Step 2 파일 올리기

생성된 저장소에서 **uploading an existing file** 링크 또는 상단 메뉴에서 [Add file] → [Upload files]를 선택합니다.

코워크에서 받은 ZIP 파일의 압축을 푼 뒤 파일들을 드래그해서 업로드하고 [Commit changes]를 누르면 업로드가 완료됩니다.



파일 업로드



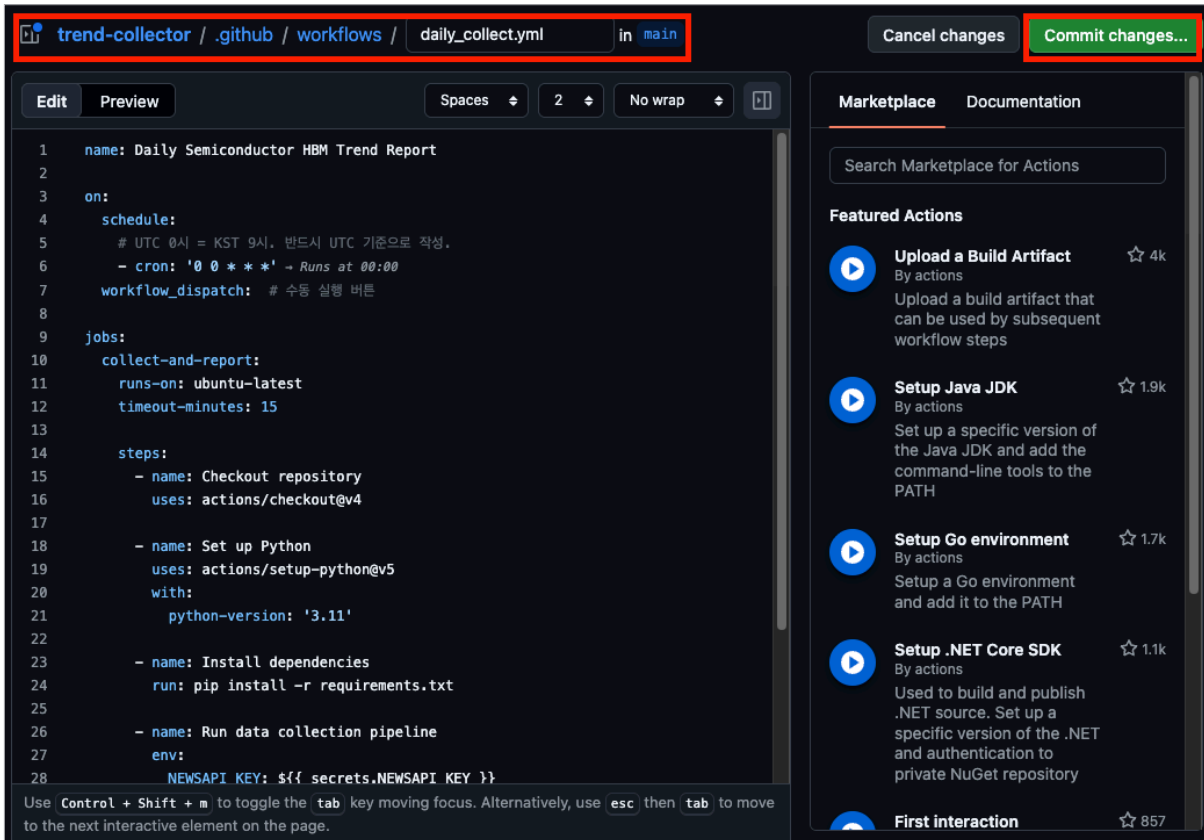
업로드하기 전에 config.yaml 파일을 한 번 확인해 보세요. 정상적으로 생성되었다면 슬랙 웹훅 URL은 비어 있으며 “GitHub Secrets 또는 환경 변수로 주입. 여기에 직접 URL을 넣지 마세요” 같은 안내만 들어 있습니다. 만약 실제 웹훅 URL이 적혀 있다면 반드시 삭제한 뒤 업로드하세요. URL이 깃허브에 올라가면 다른 사람이 여러분의 슬랙 채널로 메시지를 보낼 수 있습니다.

워크플로 파일은 예외입니다

깃허브 웹 화면에 드래그해서 올리면 폴더 구조가 유지되지 않고 모든 파일이 저장소 최상위(루트)에 나란히 올라갑니다. 파이썬 파일은 루트에 있어도 되지만, 워크플로 파일인 `daily_collect.yml`은 반드시 다음 위치에 있어야 합니다.

```
.github/workflows/daily_collect.yml
```

이 경로가 아니면 [Actions] 탭에서 워크플로가 보이지 않고 깃허브 액션이 인식하지 못합니다.



workflow 파일 등록

가장 쉬운 방법은 나머지 파일을 먼저 업로드한 뒤, [Add file] → [Create new file]을 선택하여 파일 이름에 `‘.github/workflows/daily_collect.yml’`이라고 경로까지 입력하는 것입니다. 슬래시(/)를 입력할 때마다 자동으로 폴더가 생성됩니다. 그다음 `daily_collect.yml` 파일의 내용을 복사해서 붙여넣고 [Commit changes]를 누르면 정상적으로 등록됩니다.

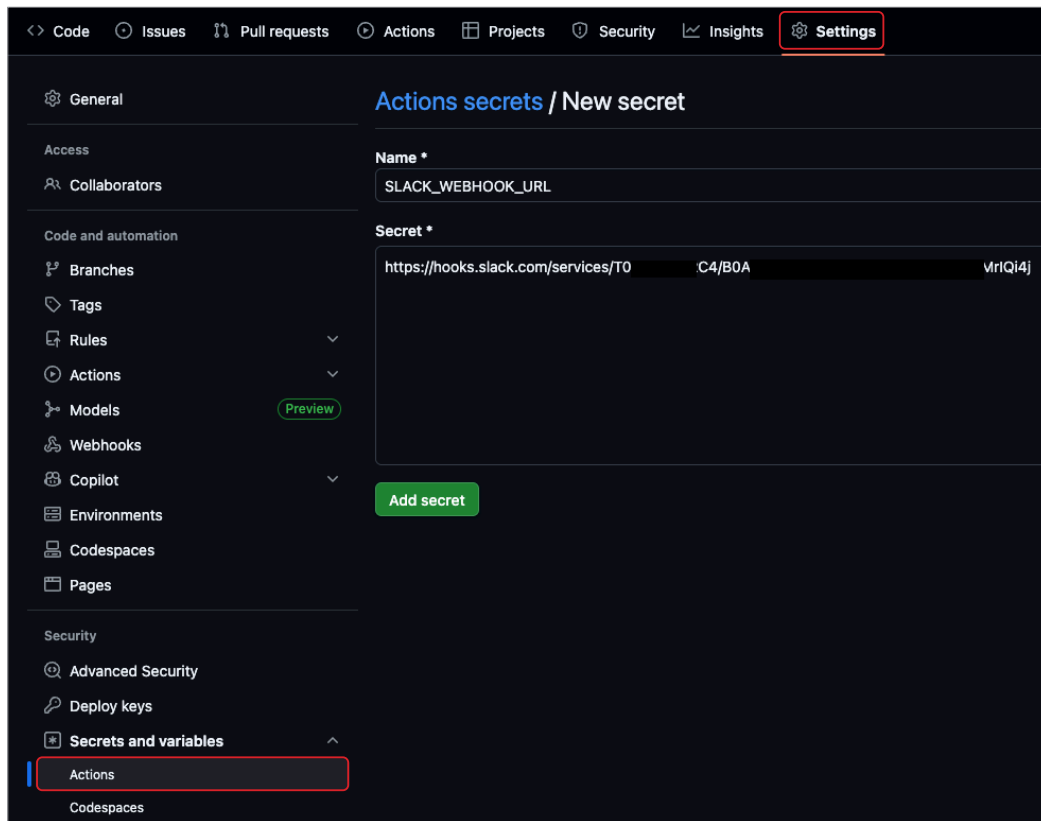
Step 3 - Secrets 등록하기

슬랙 알림이나 API를 사용할 예정이라면 [Settings] → [Secrets and variables] → [Actions]로 이동합니다. 이 단계는 슬랙 웹훅 URL이나 API 키를 준비한 경우만 해당합니다. 둘 다 없다면 이 단계를 건너뛰어도 자동화는 정상 동작합니다.

[New repository secret]을 눌러 다음과 같은 값을 추가합니다. 이름은 정확히 다음과 같이 입력해야 합니다. 대소문자를 틀리면 코드가 값을 찾지 못합니다.

이름(Name)	값(Secret)	필수 여부
SLACK_WEBHOOK_URL	앞에서 만든 슬랙 웹훅 URL	슬랙 알림을 원하면 필수
NEWSAPI_KEY	newsapi.org에서 발급받은 API 키	선택(더 풍부한 뉴스 데이터 제공)
YOUTUBE_API_KEY	구글 클라우드에서 발급받은 YouTube Data API 키	선택
FRED_API_KEY	FRED에서 발급받은 API 키	선택(미국 경제지표 분석 시 필요)
DATA_GO_KR_KEY	공공데이터포털에서 발급받은 API 키	선택(부동산 실거래가 활용 등에 필요)
STEAM_API_KEY	Steam에서 발급받은 API 키	선택(게임 분야 분석 시 필요)

API 키를 config.yaml에 직접 적지 않고 GitHub Secrets에 저장하는 이유는 보안 때문입니다. Secrets에 저장한 값은 실행 시 환경 변수로 전달되고 화면에서는 ****로 표시됩니다. 코드 저장소에 민감한 정보가 남지 않아 훨씬 안전합니다. 또한 slack_notifier.py는 SLACK_WEBHOOK_URL 환경 변수를 먼저 읽도록 만들어져 있으므로 config.yaml은 비워 둔 상태로 사용하면 됩니다.



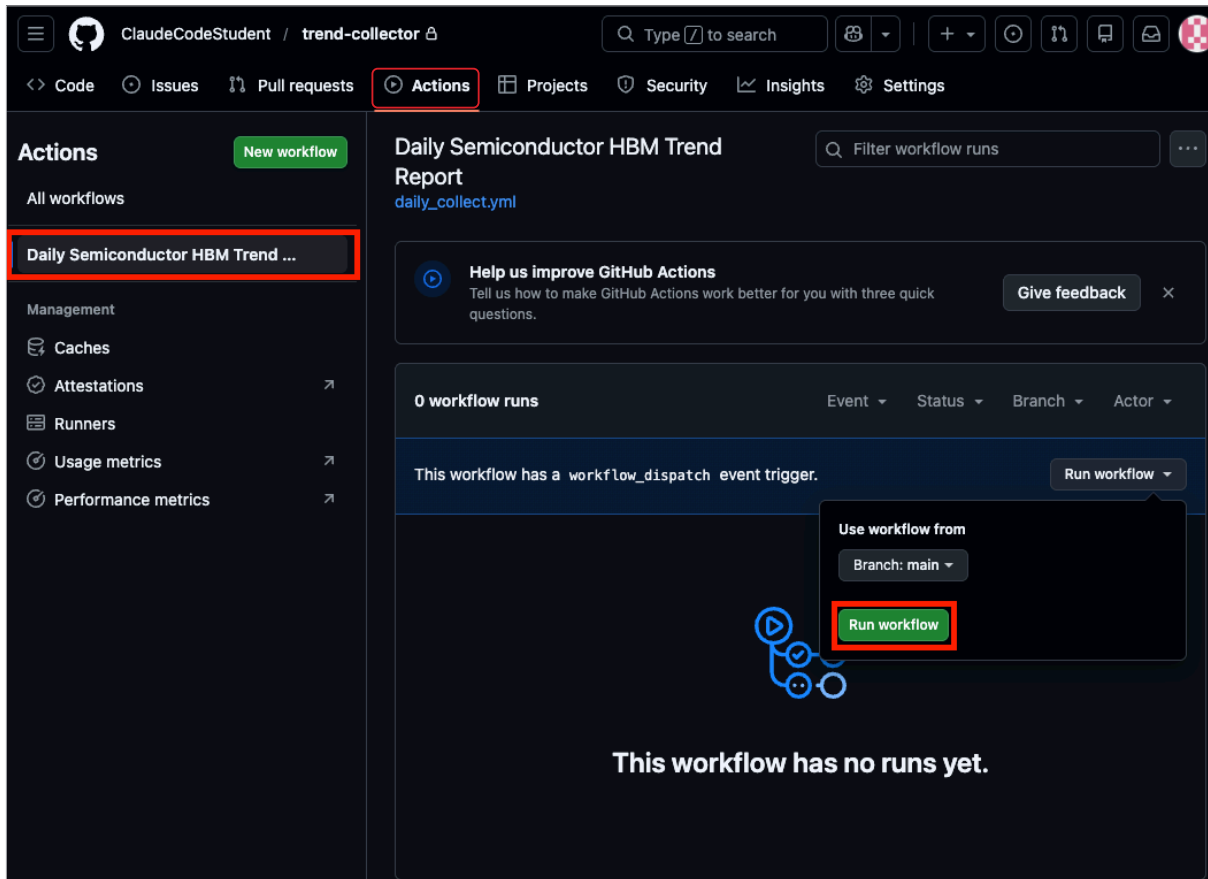
GitHub Secrets 등록 화면



GitHub Secrets는 저장 후 다시 내용을 확인할 수 없습니다. 잘못 입력했다면 삭제한 뒤 새로 만들어야 합니다.

Step 4 워크플로 활성화 및 테스트

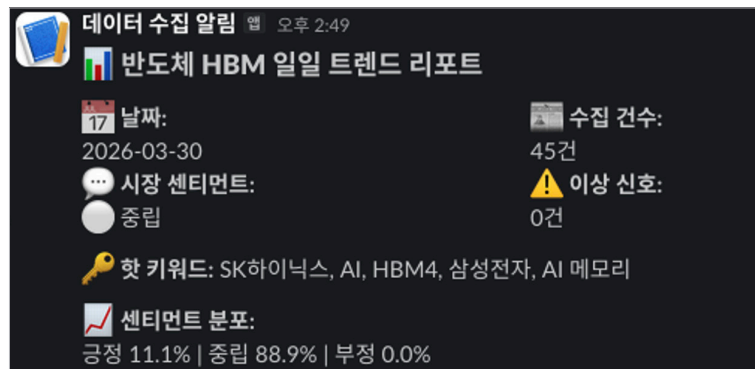
저장소의 [Actions] 탭을 열면 등록된 워크플로가 나타납니다.



깃허브 [Actions] 탭, 워크플로 수동 실행 후 초록색 체크가 표시된 화면

워크플로를 선택한 뒤 [Run workflow]를 클릭하면 즉시 한 번 실행하여 테스트할 수 있습니다. 1~5분 정도 지나 초록색 체크 표시가 나타나면 성공입니다. 이후부터는 예약된 시간마다 자동으로 실행됩니다.

슬랙 웹훅을 등록했다면 실행이 끝난 직후 슬랙 채널로 알림이 전송됩니다. 수집 건수, 감성 분석, 주요 키워드, 핵심 뉴스가 포함된 메시지가 도착하면 모든 설정이 완료된 것입니다.



슬랙 채널에 도착한 알림 메시지(요약 내용 포함)

지금까지 만든 시스템을 한 장으로 정리하면 다음과 같습니다. 두 개의 실행 방식(코워크 스케줄과 깃허브 액션)이 같은 프로젝트를 실행하고, 결과가 슬랙 알림으로 도착하는 구조입니다.



data-collector 자동화 파이프라인

여기까지 오면 코워크에 한 문장을 입력하는 것에서 출발해, 여러분이 잠든 사이에도 데이터를 수집하고 분석 결과를 전달하는 자동화 시스템이 완성됩니다.